5/8/24

EX6: Error Detection and Correction using Hamming Code

AIM: To write a program to implement error detection and correction using Hamming code

ALGORITHM:

Step 1: Start the program.

Step 2: Read No. of 4-bit data blocks (n) forming input data stream.

Step 3: For each 4 bit data block D = d1 d2 d3 d4, perform Hamming encoding.

a. Arrange 7 bits with parity bits at positions 1, 2, 4 and data bits at positions 3, 5, 6, 7.

b. Compute parity bits using even parity:

P1 = d1 XOR d2 XOR d4 - 1, 3, 5, 7

P2 = d1 XOR d3 XOR d4 - 2, 3, 6, 7

P4 = d2 XOR d3 XOR d4 - 4, 5, 6, 7

c. Form the 7-bit Hamming code - P1 P2 d1 P4 d2 d3 d4

Step 4: Simulate transmission by introducing a single-bit error at a chosen-bit position in code word.

Step 5: At receiver, recompute check bits from the received code word,

C1 = P1 XOR d1 XOR d2 XOR d4

C2 = P2 XOR d1 XOR d3 XOR d4

C4 = P4 XOR d2 XOR d3 XOR d4.

Step 6: Compute Error posibon = 14 C2 C1

OUTPUT:

```
=== Hamming Code - Error Detection and Correction ===
Enter number of 4-bit data blocks in the stream: 2

-- Block 1 --
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits              : 1 0 1 1
Encoded Hamming Code (7 bits)   : 0 1 1 0 0 1 1
Enter bit position to introduce error
(1-7, 0 = no error)             : 3
Transmitted Code (with error)   : 0 1 0 0 0 1 1
Received Code                   : 0 1 0 0 0 1 1
Error detected at bit position  : 3
Corrected Code                  : 0 1 1 0 0 1 1
Recovered Data Bits             : 1 0 1 1

-- Block 2 --
Enter 4 data bits (space separated, e.g. 1 0 1 1) : 0 1 1 0
Original Data Bits              : 0 1 1 0
Encoded Hamming Code            : 1 1 0 0 1 1 0
Enter bit position to introduce error : 6
Transmitted Code                : 1 1 0 0 1 0 0
Received Code                   : 1 1 0 0 1 0 0
Checking for errors...
Error detected at bit position  : 6
Corrected Code                  : 1 1 0 0 1 1 0
Recovered data Bits             : 0 1 1 0
```

Step 7: If error position = 0, no error, else the bit at "Error Position" is inverted (complemented).

Step 8: Extract the original 4 data bits from the corrected 7-bit code word.

Step 9: Repeat steps 3 to 8 for all n blocks in data stream.

Step 10: Stop.

PROGRAM:

```
def encode(d):
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f"Error at bit pos. {error_pos} -> Correcting")
        h[error_pos-1] ^= 1
    else:
        print("No error detected")
    return error_pos
```

```
def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("=== Hamming Code - Error detection & Correction ===\n")
    n = int(input("Enter No of 4 bit data blocks: "))
    for b in range(1, n+1):
        print(f"\n--- Block {b} ---")
        d = list(map(int, input("Enter 4 data bits (space seperated, e.g. 1011): ").split()))

        h = encode(d)
        print(f"Original    : {to_str(d)}")
        print(f"Encoded HC  : {to_str(h)}")

        pos = int(input("Enter bit position to introduce error (1-7, 0, =no error): "))

        if 1 <= pos <= 7:
            h[pos-1] ^= 1
            print(f"Transmitted code (with error {pos}): {to_str(h)}")

        else:
            print(f"Transmitted (no error induced): {to_str(h)}")
        print(f"Recieved code: {to_str(h)}")
        detect_and_correct(h)
        print(f"Corrected code  : {to_str(h)}")
        rd = [h[2], h[4], h[5], h[6]]
        print(f"Recovered Data bits : {to_str(rd)}")

if __name__ == "__main__":
    main()
```

RESULT:

The program to implement error detection and correction using Hamming code concept was successfully written.